

About me

1. My motivation to join the club is to get more exposure to electrical engineering as I am an EE student but still don't know much about core of my department. I have experience from learning stuff due to apping for Abhuday ,Agnirath and elecfest ,all were electrical oriented positions.
2. My commitments to academics are minimal as I study right before the quizzes and mostly complete assignments or lab work over weekends so i am free every working day. I am going to app for the position of pm in inot club so there will be no toher commitments except these during my tenure. I mostly will do club work and prioritize it more expect for like quiz times.

General Questions

1. I would like to work on domains related to digital systems,embedded systems and designing of PCB's.
2.
 - a. Title: Project Talos. Overview :A wearable glove that tracks finger movements and provides physical resistance (haptic feedback)when interacting with an object in a virtual environment. It communicates with the PC/VR headset and can be use in XR.
 - b. Timeline
 - i. Weeks 1-3:component selection , making the schematic and pcb design.
 - ii. Weeks 3-6:3D printing the necessary parts and firmware development , basic game engine integration using drivers (like openVR).
 - iii. Weeks 6-7: finishing up the soldering and assembly of the hardware and the basic structure of the glove.
 - iv. Weeks 7-9:testing of the power ,latency optimization ,debugging and final touch ups.
 - c. Expected deliverables:
 - i. A fully assembled haptic XR glove prototype.
 - ii. A custom PCB integrating the MCU,motor drivers and power management.
 - iii. A basic demo where user feel different levels of haptic feedback while interacting with different things and the translation of movement from the real world to the virtual world
 - d. What does the club gain:
 - i. A very interesting visual and interactive demo piece for tech fests which can be taken to the next level if we collab with the envisage's gamedev / xr vertical.
 - ii. Teaches the members pcb designing how hardware interacts with the game engine and it also teaches about signal application in real world to the project members.
 - e. Deputy Coordinators
 - i. 2-3 deputy coordinators.

- ii. One of the dcs will work on the pcb while the other will work on the integration part of the project the extra dc can be from working together with envisage's team to get the game engine stuff done.
- f. The budget will be depending on the complexity of the project and can range from 6000-7000 Rs.

Events and PR:

1.
 - a. The best students want to join teams/clubs that are active innovative, resourceful and fun. Strong PR showcases the club's culture and technical caliber thus motivating juniors to join the club. We also need pr to get the proper funding for the club which decides the range of components we can get and also the range of competitions conducted by the club.
 - b.
 - i. We can conduct a competition based on control systems by allotting the participants an unstable physical system and making them write the firmware for it and tune the PID logic for that system. We can also include reverse engineering into this challenge to make it more interesting/hard to the participants.
 - ii. We can create a challenge where the participants are supposed to make a pcb with different constraints (thermal, mechanical, etc) like for the case of avionics. We can make this instead of a 3 hr type of challenge to a type of hackathon. We can also keep an PCB upskilling workshop type of event after which we can conduct this competition to increase more involvement.
 - c.
 - i. We can conduct the PCB workshop+ competition across the dates september 12-14th which is after quiz 1 but not right after and with enough gap for the students to recover from the burnout from quiz 1 week these three days will be enough time to keep a 24hr or 48hr hackathon on the PCB.
 - ii. Another time for events can be during Dussehra holidays since there may be students who didn't go home and have their classes canceled during that week. Events during that time can attract those students.
 - d. Yes i would be interested to work on this since to teach complex concepts in simpler terms means I should have enough expertise in that topic to be able to do that. This will make me upskill myself first which will be beneficial to me and secondly finding the connection between those research topics and connecting them everyday life in simpler way feels like u just realised something and it would be nice to share them with my peers and other students.

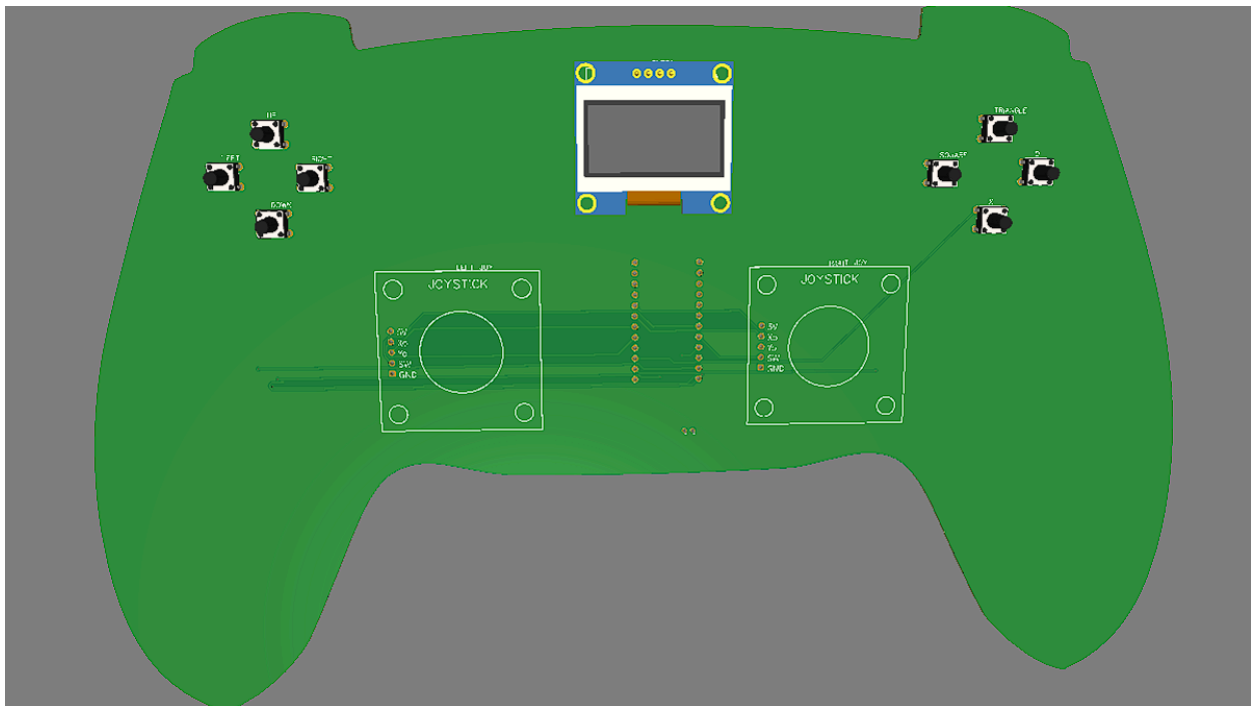
PCB Designing

1.
 - a. Potential areas where pcb designing is useful

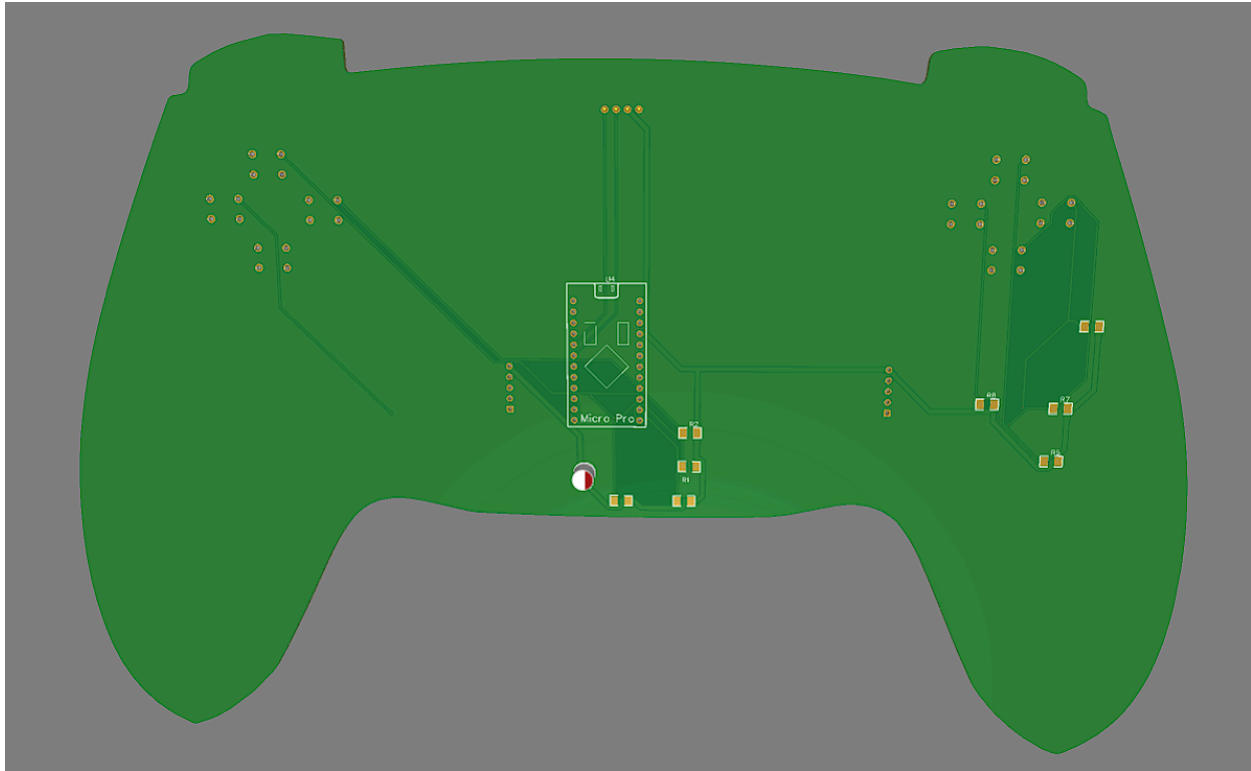
- i. In aerospace applications such as a rocket flight computer jumper wires and breadboards will instantly disconnect under launch forces. A custom PCB ensures structural integrity and allows for the integration of specific modules like LoRa telemetry transceivers RTK GPS and dual deployment pyrotechnic channels onto a single rigid board.
 - ii. For electric vehicles or solar cars managing power distribution requires exact specifications. A custom Battery Management System (BMS) or a precharge circuit needs specific copper trace widths and heavy duty contactors to handle massive inrush currents safely without causing system brownouts or thermal runaway.
 - iii. When electronics must slide into a specific aerodynamic airframe tube or fit inside a highly constrained mechanical structure a custom PCB is the only viable option.
- b. It takes about 3 months to complete a custom PCB with a team of 2-4 people with more people allowing the checks for logic and other flaws to go much faster and smoother. I can work on the PCB anytime after classes end except during the quiz week.

2.

a.  `electronic_club_app_pcb`



b.



- c.
- 3.
- Vias are useful to connect different layers so that it decreases complexity of routing.
 - Traces are the current carrying elements and their width depends on their thermal limits and the current that flows through it. It is important to keep the power traces wider because huge amount of current flows through them and it generates heat, it also helps minimizes voltage drops.
 - Ground plane reduces complexity of the routing and heat dissipation significantly.
 - To clean up the routing we can make use of vias to move the wiring from one plane to another or move the components around to get better routing .
 - Edge routed traces risk being physically severed during manufacturing panel separation and are highly vulnerable to external noise.
 - Traces running parallel and close together for long distances will likely suffer from the electromagnetic field of one trace inducing unwanted noise or false signals into the adjacent trace. If they are in different layers then they can also create a capacitance between them

PID

- A PID controller is a control loop feedback mechanism used in industrial and robotic control systems. Its job is to continuously calculate an error value $e(t)$ which is the difference between the desired state and a measured variable(present state) and apply a correction based on three distinct terms:

- a. Proportional (P) → Reacts directly to the current error. The bigger the error, the harder it pushes. If we only use P we can never reach the exact target (becomes exponential). The P term's purpose is to provide the main driving force to get to the target right now.
 - b. Integral (I) → Accumulates past errors over time. Even if the current error is tiny the I term adds it up until the controller pushes hard enough to eliminate that remaining error completely. The I term's purpose is to eliminate that steady state error.
 - c. Derivative (D) → Looks at the rate of change of the error. It acts as a damper or shock absorber. If the variable is approaching the target too fast, the D term slows the system down to prevent overshooting. The D term's purpose is to prevent the system from overshooting.
 - d. $u(t) = K_p e(t) + K_i \int_0^t e(T) dT + K_d \frac{de(t)}{dt}$
2. One of the uses of PID control systems is maintaining the target speed (cruise control) of an electric vehicle powered by an axial flux permanent magnet synchronous motor (AF-PMSM).
- a. The desired velocity of the car is set to some v (cruise speed)
 - b. The sensors read the velocity of the car at time t which is the measured variable.
 - c. The PID controller takes this error and calculates how much to adjust the motor's power electronics to get to cruising speed.
 - d. The relation governing the motion of the vehicle is →
 - e. $m \frac{du(t)}{dt} = K_p e(t) + K_i \int_0^t e(T) dT + K_d \frac{de(t)}{dt}$
 - f. Where $e(t) = v - u(t)$ (if initial speed is less than v) or $e(t) = u(t) - v$ (if initial speed is higher than v)
3. The methods used to tune PID are:
- a. Manual Tuning: This is the most common approach, where gains are adjusted live based on observing the system's reaction.
 - b. Ziegler Nichols Method: A structured practical method that deliberately pushes the system into continuous oscillation to calculate a mathematical baseline for the gains.
 - c. Cohen Coon Method: Similar to Ziegler Nichols, but specifically optimized for systems that have a significant time delay before they react to an input.
 - d. Auto Tuning: Using software algorithms (like relay feedback) to automatically test the system and calculate the optimal parameters without human intervention.
 - e. Relay Feedback Auto Tuning :
 - i. Instead of using a PID equation the microcontroller temporarily disconnects the PID logic and replaces it with a simple relay on/off mechanism that makes the system oscillate.
 - ii. The controller looks at the target setpoint and if the system is below the target it turns the motor 100% ON. The system accelerates and eventually crosses above the target line. The moment it crosses the relay turns the motor 100% OFF
 - iii. The system coasts upward loses momentum, and falls back down. The moment it crosses below the target line again the relay turns it back to 100% ON. This creates a continuous stable oscillation around the target setpoint.

- iv. While the system is bouncing safely up and down the microcontroller's software measures two things from the resulting wave it's amplitude and ultimate period the microcontroller finds ultimate gain using the formula $K_u = 4d/(\pi)a$.
- v. Once the microcontroller has computed K_u and T_u from the relay test it plugs those numbers into the standard Ziegler Nichols formulas and then saves the resulting K_p K_i K_d values and reverts back to PID logic with these values.

4.

- a. The robot is experiencing a steady-state error. It is getting stuck 0.11m short of its goal because the Proportional (P) term decreases as the robot gets closer to the target . Eventually, the error is so small that the P term's force drops below the physical friction threshold of the robot's wheels thus causing it to stop short. By increasing the Integral (I) term the controller will start a mathematical stopwatch. It continuously adds up that 0.11m error over time. Even though the error is tiny, the accumulated sum grows larger and larger as time increases until it generates the exact extra force needed to overcome friction and pull the robot to precisely 10.00m.
- b. The robot's response is highly underdamped. It has too much momentum when it arrives at the 10m mark, causing it to overshoot then it detects that it went too far and reverses too aggressively, undershooting and repeat these oscillations. Increasing the Derivative (D) gain introduces a damping force based on the rate of change of the error. As the robot rapidly approaches 10m, the D term sees the error shrinking fast and applies a opposing force to slow the robot down allowing it to glide smoothly onto the 10m mark and settle immediately without bouncing back and forth.
- c. When the robot is at 0m and the target is 10m, the initial error is massive (10m). The Proportional term is supposed to look at this present error and commands aggressive acceleration. If the robot is very slow the P term is too low meaning the controller isn't translating that massive 10m error into a powerful enough force. Increasing K_p will make the robot rush out of the starting point with much higher velocity getting it to the 10m mark significantly faster.

Digital Systems

1.

- a. XOR gate
 - i. $Y = A \text{ XOR } B$

A	B	Y
0	0	0
0	1	1

1	0	1
1	1	0

- ii. Whenever we have odd no of 1 inputs we get the output as 1 otherwise the output is 0.

b. NAND gate

- i. $F=A \text{ AND } B \rightarrow Y=F'$

A	B	F	Y
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

- ii. Whenever the inputs all are 1's then we get output as 1 otherwise the output is 0.

2.

- a. Gate level minimization is the process of reducing a complex Boolean expression to its simplest possible form. In digital systems simpler logic expressions means we get hardware that uses fewer logic gates and fewer connections. This minimization is critical because it minimizes the propagation delay of signals passing through the system.
- b. A Karnaugh Map is a grid based method used to simplify Boolean expressions without relying on algebraic manipulation. Instead of using a standard truth table, a K-map transfers the output values into a two dimensional grid where each cell represents a specific combination of inputs (minterm).
- c. The fundamental reason K-maps work is that they are structured using Gray code sequence. Unlike standard binary counting, Gray code ensures that any two adjacent cells whether placed horizontally, vertically, or around the edges differ by exactly one variable. Because adjacent cells only differ by a single variable, grouping them uses the Boolean algebra theorem $A+A'=1$. When we loop adjacent cells together, the variable that changes state between those cells cancels itself out, leaving only the variables that remain constant. This cancellation systematically removes redundancies thus yielding the minimized logic circuit.

d. Task 1

- i. The logisim code file is in the git hub repository:
https://github.com/Devesh-Ruthvik-Yarlagadda/ee25b042_COORDINATOR-APPLICATIONS-2026-27/tree/Devesh_ee25b042
- ii. We need to take 3 flip flops since the number of states is 8. The truth table for the output is as follows

a	b	c	D ₀	D ₁	D ₂	a ⁺	b ⁺	c ⁺
---	---	---	----------------	----------------	----------------	----------------	----------------	----------------

0	0	0	1	1	0	0	1	1
0	0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0	1
0	1	1	0	1	0	0	1	0
1	0	0	1	1	1	1	1	1
1	0	1	0	0	1	1	0	0
1	1	0	1	0	0	0	0	1
1	1	1	0	1	1	1	1	0

iii. Clearly $D_0 = c'$

iv. The karnaugh map for D_1 gives $D_1 = b \text{ XNOR } c$

1	0	1	0
1	0	1	0

v. The karnaugh map for D_2 gives $D_2 = (b \text{ XOR } c') \text{ AND } a$ i.e $D_2 = ab' + ac + a'bc'$

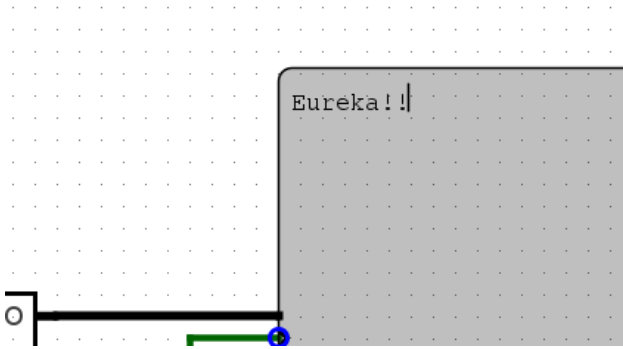
0	0	0	1
1	1	1	0

vi. We connected the same clock for the counter and the verifier to make it synchronous. We connect the reset pin of the verifier to reset pin of the last two flip flops and to the set pin of the first flip flop. This resets the original value of the counter to 001 and also resets the verifier and the counter is set to 001 because the verifier checks for binary 1 first not binary 0.

vii. Due to propagation delay the outputs of three flip flops do not change at the same time at lower tick rates but do so at higher tick rates(4 tick for high).The verifier still shows correct output once we change the reset to 1 and back to 0. This makes the whole circuit synchronous and it still works at lower ticks because the clock's rising edge occurs after the outputs of the flip flops come.

e. Taks2

i. The hexadecimal value for the flag is 0x008004000000002D to get Eureka!! printed.



ii.

iii. The below is the solution process that i used to get the solution

Handwritten solution process for a system of linear equations:

$$P_2 \rightarrow 0-7+8-15 \Rightarrow 0 \times 101101 \quad \text{P1✓}$$

$$P_3 \rightarrow \text{xor} \rightarrow \text{not} \quad \text{P2✓}$$

$$P_1 \rightarrow \begin{matrix} 23-16 \rightarrow 7-4 \rightarrow \text{AND} \\ 31-24 \rightarrow 7-4 \rightarrow \text{Combine} \\ 3-0 \rightarrow 3-0 \\ 3 \rightarrow 2-2 \end{matrix} \quad \text{P3✓}$$

$$\begin{matrix} 2-7-350+7-0 \\ 5-2-2 \rightarrow 0-0 \rightarrow \text{P5✓} \end{matrix}$$

$$\begin{matrix} \text{not equal} \\ 7-0 \rightarrow 7-0 \\ 15-8 \rightarrow 7-0 \\ 15-8 \rightarrow 7-0 \end{matrix} \quad \text{P4✓}$$

$$\left[\begin{pmatrix} 3-0 \\ 45-16 \\ 13-0 \end{pmatrix} + \begin{pmatrix} 3-0 \\ 2-0 \\ 7-0 \end{pmatrix} \right]_2 = \left[\begin{pmatrix} 7-0 \\ 63-36 \end{pmatrix} \right]_2$$

iv.

Handwritten solution process for a system of linear equations:

$$B_2 \rightarrow 0 \rightarrow 00$$

$$B_1 \rightarrow 0010101 \rightarrow 2D$$

$$B_3 \rightarrow 00000000 \rightarrow 00$$

$$B_4 \rightarrow 00000000 \rightarrow 00$$

$$B_5 \rightarrow 00000000 \rightarrow 00$$

$$B_6 \rightarrow 00000000 \rightarrow 00$$

$$B_7 \rightarrow 00000000 \rightarrow 00$$

$$B_8 \rightarrow 00000000 \rightarrow 00$$

v.

- vi. I took bits which don't have any relevance to getting the correct inputs to the verifier as 0. I also took byte 2 as 0 since it allows me to get byte 1 from their relation to get P_2 . I similarly found the other bits with their relevant conditions.
- vii. The verifier has a mux with 8 inputs each containing 7 bits which equate to the binary equivalents of ascii values of each character of the word Eureka.
- viii. The selector pin is connected to a register's output which changes for every clock pulse counting from 000→111 total of 8 values so that the correct ascii value goes to the terminal.
- ix. The register adds 1 to its output for every clock pulse and when it is 7 the output of and gate goes high and the mux outputs 7 to the register so the output of register stays fixed at 7 and the D pin which tells the Terminal to print the letter turns low due to the D flip flop from the next cycle onwards thus stopping the printing process.
- x. The O pin gets 7 bits as output which changes from P_1 to P_8 for every clock pulse. Since only one ascii value is going as input to the terminal only one symbol gets printed. This along with the D pin makes sure that after Eureka is printed the terminal stops its process and the register output is fixed making the mux stop functioning and giving P_8 as output to O.

Hardware Interfacing : Sensors & ADC

1. If the sensor is showing output as 0.75 volts then first we subtract 0.5 volts since at that temperature we get 0°C . The voltage we get is 0.25V and when now for every degree we have an increase of 10mV so 0.25×100 give 25°C
2. Since we are given digital value of 512V we get voltage by using the formulae

$$V = (\text{digitalvalue}/1023) \times 5 = 2.5024\text{V}$$
or approximately 2.5V.
3. Task
 - a. Since the simulator doesn't have a TMP36 tool I used a potentiometer since it acts similar to the sensor in the way that they are both linearly dependent.
 - b. I used 3.3V instead of the 5V as the 3.3V is a cleaner reference compared to 5V since it is less noisy.
 - c. <https://wokwi.com/projects/464471195049520129>